SOLAR▸SWARM

Operations
Martin Pham | Perth, AU

Setup & Deployment Guide

This guide walks through setting up the Solar Swarm from scratch on a fresh machine.

Prerequisites

- Python 3.11+
- Node.js 18+ and npm (for the swarm board)
- A terminal and basic familiarity with the command line
- The following API accounts (details below):
- OpenAI (required)
- GoHighLevel (required for production; mock mode works without it)
- Slack (optional — Slack alerts and interactive buttons)
- Retell AI (optional — voice call automation)
- ElevenLabs (optional — premium voice quality)

Step 1: Clone or extract the project

```
# If using git
git clone <your-repo-url> solar-swarm
cd solar-swarm

# If you downloaded a ZIP
unzip Solar-Concept-main.zip
cd Solar-Concept-main
```

Step 2: Create a Python virtual environment

```
python3 -m venv .venv
source .venv/bin/activate   # macOS/Linux
# .venv\Scripts\activate    # Windows
```

Step 3: Install Python dependencies

```
pip install -r requirements.txt
```

This installs:

PACKAGE	PURPOSE
`openai`	GPT-4o for all AI tasks
`flask`	API server
`flask-cors`	CORS for the swarm board
`flask-limiter`	Rate limiting on auth endpoints
`apscheduler`	Scheduled agent jobs
`python-dotenv`	`.env` file loading
`requests`	HTTP calls to GHL, Retell, etc.
`pytz`	Timezone handling
`rich`	Formatted terminal output
`simple-salesforce`	Salesforce CRM support
`slack-sdk`	Slack notifications
`PyJWT`	JWT auth tokens
`bcrypt`	Password hashing

Step 4: Configure environment variables

Copy the example env file:

```
cp .env.example .env
```

Open `.env` and fill in your values:

```
# ── Required ──────────────────────────────────────────────
OPENAI_API_KEY=sk-...           # Get from platform.openai.com

# ── GoHighLevel (primary CRM) ─────────────────────────────
GHL_API_KEY=                    # Settings → Integrations → API → Private Integration Key
GHL_LOCATION_ID=                # Settings → Business Profile → Location ID in URL
GHL_PIPELINE_ID=                # CRM → Pipelines → pipeline ID from URL
GHL_WEBHOOK_SECRET=             # Settings → Integrations → Webhooks → Signing Secret

# ── Slack (optional — for notifications and interactive buttons) ─
SLACK_WEBHOOK_URL=              # Create at api.slack.com/apps → Incoming Webhooks
SLACK_SIGNING_SECRET=           # App credentials → Signing Secret (for /slack/actions)

# ── Voice AI (optional) ───────────────────────────────────
RETELL_API_KEY=                 # app.retellai.com → API Keys
ELEVENLABS_API_KEY=             # elevenlabs.io → Profile → API Key
ELEVENLABS_DEFAULT_VOICE=       # Voice ID from ElevenLabs dashboard

# ── Capital ───────────────────────────────────────────────
WEEKLY_BUDGET_AUD=500           # Total weekly experiment budget

# ── Security ──────────────────────────────────────────────
GATE_API_KEY=                   # Choose any strong random string for the Human Gate API

# ── Ports ─────────────────────────────────────────────────
PORT_HUMAN_GATE=5000
PORT_GHL_WEBHOOKS=5001
PORT_VOICE_WEBHOOK=5002
PORT_DASHBOARD_API=5003

# ── HubSpot (optional fallback CRM) ───────────────────────
HUBSPOT_API_KEY=

# ── Salesforce (optional fallback CRM) ────────────────────
SALESFORCE_USERNAME=
SALESFORCE_PASSWORD=
SALESFORCE_SECURITY_TOKEN=
SALESFORCE_CLIENT_ID=
SALESFORCE_CLIENT_SECRET=
```

The system works with only `OPENAI_API_KEY` set. All other integrations fall back to mock/demo mode when their keys are missing.

Step 5: Initialise the database

```
python3 -c "from memory.database import init_db; init_db()"
```

This creates `swarm.db` in the project root with all tables. The `DATABASE_PATH` env var overrides the default location.

Step 6: Create required directories

```
mkdir -p proposals reports memory/knowledge public
```

DIRECTORY	PURPOSE
`proposals/`	Generated proposal text files
`reports/`	Weekly client report files
`memory/knowledge/`	Warm memory JSON files
`public/`	`board-state.json` for the swarm board

Step 7: Create board-state.json

```
echo '{}' > public/board-state.json
```

Step 8: Test the setup

Run a quick qualification test to verify OpenAI and the database are working:

```
python3 cli.py test-lead
```

Expected output: a qualification result with a score between 1 and 10.

Step 9: Start the system

```
python3 main.py
```

This starts:

- The General scheduler (runs every 6 hours, first run immediately)
- All data collection and research jobs on their scheduled intervals
- Four Flask servers on ports 5000–5003

You should see output like:

```
[DB] Database initialised — swarm.db
[MAIN] Starting Human Gate API on port 5000
[MAIN] Starting GHL Webhook Server on port 5001
[MAIN] Starting Voice Webhook Server on port 5002
[MAIN] Starting Dashboard API on port 5003
[SCHEDULER] All jobs scheduled. Running.
```

Step 10: Verify all services are healthy

```
curl http://localhost:5000/health
curl http://localhost:5001/health
curl http://localhost:5002/voice/health
curl http://localhost:5003/api/health
```

All should return `{"status": "ok", ...}`.

Step 11: Set up the Swarm Board (optional)

```
cd swarm-board
npm install
npm run dev
```

Open `http://localhost:5173` in a browser. The Board tab works immediately (localStorage). The Overview tab shows live data once the Python backend is running.

Step 12: Configure GHL Webhooks (production)

Your server must be publicly accessible for GHL to send events. Use ngrok for development:

```
ngrok http 5001
# Copy the HTTPS URL, e.g. https://abc123.ngrok.io
```

In GHL:

1. Go to **Settings → Integrations → Webhooks → Add Webhook**
2. Set URL to `https://abc123.ngrok.io/webhook/new-lead`
3. Select event: **Contact Created**
4. Add another webhook for **Opportunity Stage Change** → `https://abc123.ngrok.io/webhook/stage-change`

5. Copy the Signing Secret and add to `.env` as `GHL_WEBHOOK_SECRET`

For form submissions:

- Create a GHL Funnel with a form, set webhook URL to `/webhook/form-submit`

For voice call events:

- Point **Conversation Status Changed** to `/webhook/call-complete`

Restart `main.py` after updating `.env`.

Step 13: Set up Voice AI (optional)

```
from voice.retell_client import setup_client_voice_agent

result = setup_client_voice_agent(
    client_id="sunpower_perth",
    company_name="SunPower Perth",
    phone_number="+61892345678",          # E.164 format
    webhook_base_url="https://your-server.com",
    elevenlabs_voice_id="your-voice-id",  # Optional
)
print(result)
# {'success': True, 'agent_id': '...', 'phone': '...', 'phone_linked': True}
```

This creates the Retell agent, links the phone number, and saves the `agent_id` to `company_profiles`.

First Experiment Cycle

After starting `main.py`, the General runs within its first 6-hour window. It will:

1. Generate 3 experiment ideas using GPT-4o
2. Score each with the 4-component confidence system
3. Red-team each idea
4. Route based on adjusted score:

- Score > 8.5 → auto-approved, budget allocated
- Score 5.0–8.5 → Slack alert sent, awaiting your approval
- Score < 5.0 → auto-killed

To trigger the General immediately without waiting:

```
python3 cli.py run-general
```

To see pending experiments:

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/pending
```

Running in Production

For production deployment, use a process manager to keep `main.py` running:

systemd (Linux):

```
[Unit]
Description=Solar Swarm
After=network.target

[Service]
WorkingDirectory=/opt/solar-swarm
ExecStart=/opt/solar-swarm/.venv/bin/python3 main.py
Restart=always
User=ubuntu
EnvironmentFile=/opt/solar-swarm/.env

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable solar-swarm
sudo systemctl start solar-swarm
```

For the swarm board in production:

```
cd swarm-board && npm run build
# Serve the dist/ folder with nginx or any static host
```

Set `FRONTEND_URL` in `.env` to your deployed frontend URL so the Dashboard API allows CORS from it.

Useful CLI Commands

```
python3 cli.py test-lead              # Test lead qualification
python3 cli.py run-general            # Trigger The General immediately
python3 cli.py run-scout              # Trigger the Scout agent
python3 cli.py status                 # Show swarm summary
python3 cli.py pending                # List pending experiments
python3 cli.py approve <id>           # Approve an experiment
python3 cli.py reject <id> "reason"   # Reject an experiment
```

SOLAR▸SWARM Confidential — 09 Mar 2026 v2.0